Grado Superior

Programación

T6. Principios fundamentales de la POO

Curso 2025/26

Profesor

Raúl Murillo Gallego

«Rodéate de personas que crean en tus sueños, animen tus ideas, apoyen tus ambiciones, y saquen lo mejor de ti» —Roy T. Bennet

Presentación

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
- 6.3.1. Introducción
- 6.3.2. Ámbitos Private / Public / Protected / Por Defecto
- 6.3.3. Getters & Setters
- 6.3.4. Clases envoltorio
- 6.3.5. Clase STRING
- 6.3.5. Clase STRINGBUFFER
- 6.3.6. Método FORMAT
- 6.3.7. Clases Encapsuladoras de Fechas

6.4. Herencia
- 6.4.1. Introducción
- 6.4.2. Extends
- 6.4.3. Sobreescritura de métodos: SUPERCLASE
- 6.4.4. Interface
- 6.4.5. Abstract
- 6.4.6. Final
- 6.4.7. Herencia Simple

6.5. Polimorfismo
- 6.5.1. Introducción
- 6.5.2. Tipos de polimorfismo

6.6. Lenguaje Unificado de Modelado (UML)
- 5.4.1 Diagramas de clases

T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
6.3.1. Introducción
6.3.2. Ámbitos Private / Public / Protected / Por Defecto
6.3.3. Getters & Setters
6.3.4. Clases envoltorio
6.3.5. Clase STRING
6.3.5. Clase STRINGBUFFER
6.3.6. Método FORMAT
6.3.7. Clases Encapsuladoras de Fechas
6.4. Herencia
6.4.1. Introducción
6.4.2. Extends
6.4.3. Sobreescritura de métodos: SUPERCLASE
6.4.4. Interface
6.4.5. Abstract
6.4.6. Final
6.4.7. Herencia Simple
6.5. Polimorfismo
6.5.1. Introducción
6.5.2. Tipos de polimorfismo
6.6. Lenguaje Unificado de Modelado (UML)
5.4.1 Diagramas de clases

6.1. Introducción

Introducción

Los cuatro pilares de la Programación Orientada a Objetos (POO) son **abstracción**, **encapsulamiento**, **herencia** y **polimorfismo**. A continuación se explican cada uno de una manera sencilla:

1. Abstracción:

La abstracción se trata de simplificar cosas complejas mostrándolas de manera fácil de entender. En POO, abstraemos lo esencial de un objeto, resaltando solo lo importante.

2. Encapsulamiento:

El encapsulamiento significa ocultar los detalles internos y exponer solo lo necesario. Permite controlar cómo se accede y modifica la información de un objeto.

3. Herencia:

La herencia permite reutilizar código al crear una nueva clase basada en una clase existente. Imagina que tienes una plantilla que usas para crear nuevas cosas con base en lo que ya existe.

4. Polimorfismo:

El polimorfismo permite usar un mismo nombre de método en distintas clases para hacer cosas diferentes. Esto facilita que el código sea más flexible y adaptable.

T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
6.3.1. Introducción
6.3.2. Ámbitos Private / Public / Protected / Por Defecto
6.3.3. Getters & Setters
6.3.4. Clases envoltorio
6.3.5. Clase STRING
6.3.5. Clase STRINGBUFFER
6.3.6. Método FORMAT
6.3.7. Clases Encapsuladoras de Fechas
6.4. Herencia
6.4.1. Introducción
6.4.2. Extends
6.4.3. Sobreescritura de métodos: SUPERCLASE
6.4.4. Interface
6.4.5. Abstract
6.4.6. Final
6.4.7. Herencia Simple
6.5. Polimorfismo
6.5.1. Introducción
6.5.2. Tipos de polimorfismo
6.6. Lenguaje Unificado de Modelado (UML)
5.4.1 Diagramas de clases

6.2. Abstracción

Abstracción

La **abstracción** es un proceso de interpretación y diseño que implica reconocer y enfocarse en las características importantes de una situación u objeto, y filtrar o ignorar todas las particularidades no esenciales. Es decir, la **abstracción** trata de simplificar cosas complejas mostrándolas de manera fácil de entender.

- Dejar a un lado los detalles de un objeto y **definir las características específicas de éste**, aquellas que lo distingan de los demás tipos de objetos.

- Hay que **centrarse en lo que es y lo que hace un objeto**, antes de decidir cómo debería ser implementado.

- Se hace énfasis en el **qué hace** más que en el cómo lo hace.

Abstracción

Ejemplo: Imaginemos que queremos aplicar la abstracción a las aves:

Objeto: **Pajaro**

Características:

- Pico
- Alas
- Patas
- Plumas

Funcionalidades:

- Volar
- Picar
- Aterrizar

Ejemplo: Imaginemos que queremos aplicar la abstracción a los coches:

Objeto: **Coche**

Características:

- Modelo
- Color
- Marca

Funcionalidades:

- Arrancar
- Frenar

Nota: En una clase Coche, tenemos métodos como arrancar() o frenar(), sin necesidad de conocer todos los componentes internos que hacen posible el funcionamiento de un coche.

Abstracción

Si tenemos que definir que es una canción, no es necesario incluir todas las características, debemos de “abstraernos”, añadiendo solamente las necesarias en función del objetivo que tengamos planteado. No es lo mismo definir una clase para una Biblioteca de música que para una web de música

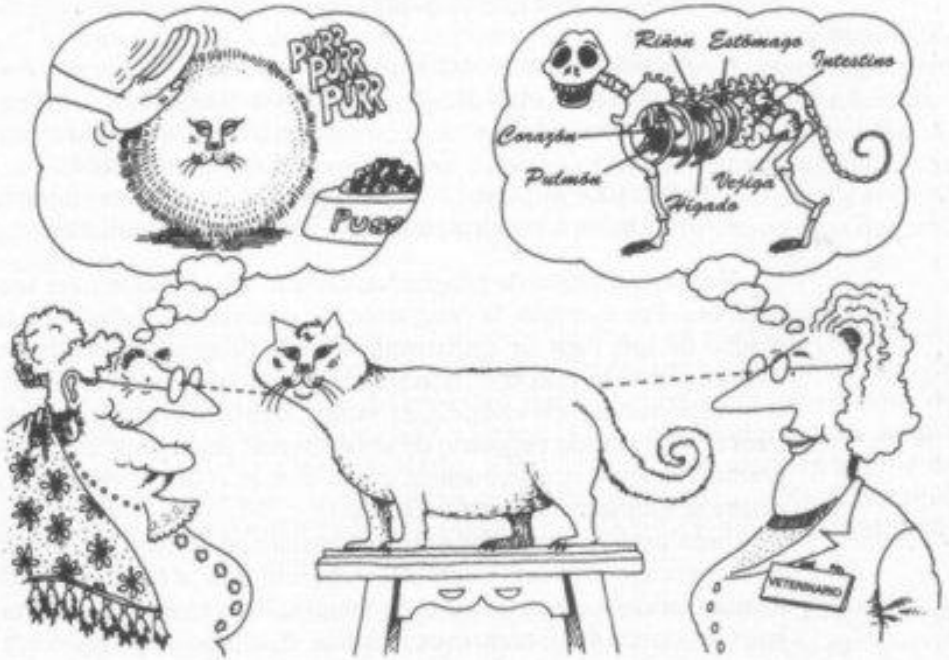
BIBLIOTECA DE MÚSICA

Cancion
- Nombre - Artista - Duración - Genero
+ Constructor() + Escuchar()

WEB DE VENTAS

Cancion
- Nombre - Artista - Precio
+ Constructor() + Comprar()

Abstracción



La abstracción se centra en las características esenciales de algún objeto, en relación a la perspectiva del observador.

T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
6.3.1. Introducción
6.3.2. Ámbitos Private / Public / Protected / Por Defecto
6.3.3. Getters & Setters
6.3.4. Clases envoltorio
6.3.5. Clase STRING
6.3.5. Clase STRINGBUFFER
6.3.6. Método FORMAT
6.3.7. Clases Encapsuladoras de Fechas
6.4. Herencia
6.4.1. Introducción
6.4.2. Extends
6.4.3. Sobreescritura de métodos: SUPERCLASE
6.4.4. Interface
6.4.5. Abstract
6.4.6. Final
6.4.7. Herencia Simple
6.5. Polimorfismo
6.5.1. Introducción
6.5.2. Tipos de polimorfismo
6.6. Lenguaje Unificado de Modelado (UML)
6.6.1 Diagramas de clases

6.3. Encapsulamiento

Encapsulamiento

El lenguaje de programación Java implementa el concepto de **encapsulación** mediante los modificadores de visibilidad que permiten establecer contenedores (paquete y clase) y a los elementos en ellos contenidos (atributos y métodos). De esta manera es posible ocultar los campos (atributos) de un objeto para que únicamente sea posible acceder a ellos mediante los métodos definidos para tal propósito.

Esta encapsulación facilita la creación de **librerías** con métodos e interfaces bien definidos, **favoreciendo la reutilización del código**.

Los modificadores de visibilidad determinan si un miembro de una clase puede ser accedido por elementos de otras clases. En Java existen 2 niveles de control de acceso sobre los que establecer modificadores de visibilidad: *nivel superior* (clases) y *nivel de miembro (atributos y métodos)*

Ámbitos Private / Public / Protected / Por Defecto

NIVELES DE CONTROL DE ACCESO

Nivel Superior (Clase)

public	Clase visible para todas la clases
package-private (sin modificador)	Clase visible para clases del mismo paquete

Nivel Miembro (Atributos y métodos)

Modificador	Clase (misma)	Clases y subclases del Paquete (mismo)	Subclase (distinto paquete)	Otros (fuera del paquete)
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
package-private (sin modificador)	Sí	Sí	No	No
private	Sí	No	No	No

Encapsulamiento

Descripción de los Modificadores:

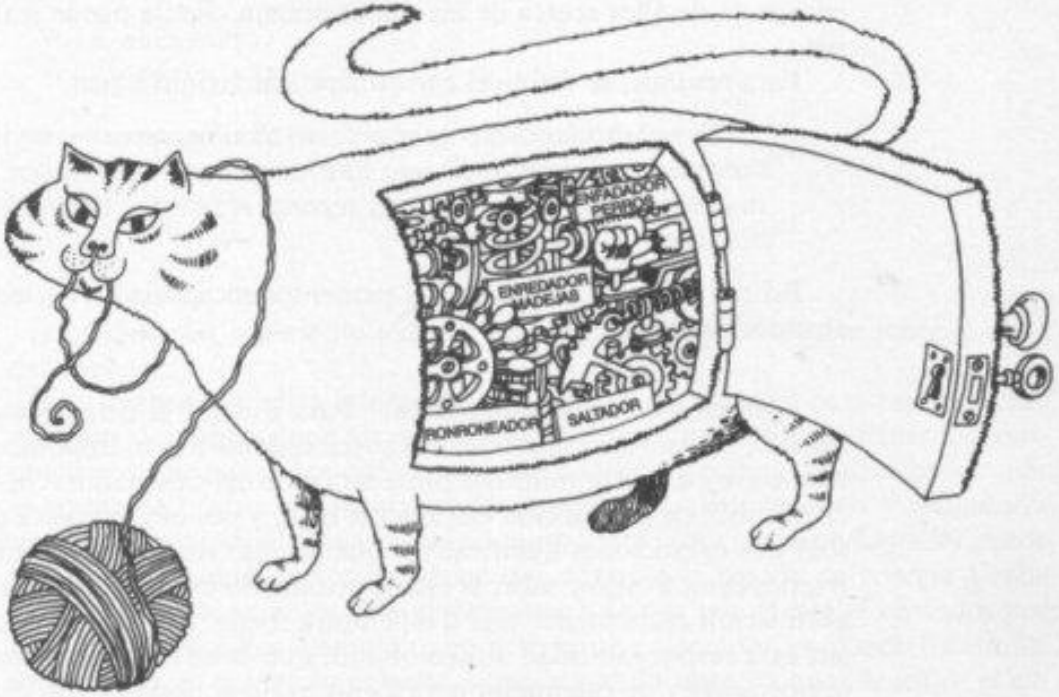
1. **public**: El miembro es accesible desde cualquier clase, sin importar el paquete.

2. **protected**: El miembro es accesible desde clases del mismo paquete y desde subclases en otros paquetes.

3. **sin modificador (Package-Private):** El miembro es accesible solo dentro del mismo paquete.

4. **private**: El miembro es accesible únicamente dentro de la misma clase.Estos modificadores son esenciales para controlar el acceso a los atributos y métodos de una clase, y establecer qué partes del código son visibles o están ocultas.

Encapsulamiento



El encapsulamiento oculta los detalles de la implementación de un objeto.

Setters y Getters

Ya se han tratado con anterioridad, que la aplicación del principio de encapsulamiento, implica dejar inaccesibles los atributos de una clase, desde el “exterior” de la misma. A tal efecto, se adoptan las siguientes medidas:

1. Aplicar el modificador de ámbito `private` a los atributos de la clase, en un intento de que los objetos de dicha clase se conviertan en “herméticos”, en lo que al acceso a los atributos desde el “exterior” se refiere. Entendiendo por “exterior” a cualquier otra clase que no sea la que “acoge” a los atributos.

2. Dotar a la clase de métodos específicos para el acceso indirecto a los atributos desde el “exterior”

Tal es la generalización de este principio, y la adopción de dichas medidas, que se ha estandarizado el nombre que deben tener estos métodos específicos, implementados únicamente para permitir acceso indirecto desde el “exterior”, a los atributos. A este “colectivo” de métodos se les llama “setters” y “getters”. De tal modo, que a cada atributo se le dota de su correspondiente “set” y su correspondiente “get”.

Clase (getter)

Un **getter** (también conocido como **accesor**) es un método que se utiliza para **obtener** o **consultar** el valor de un atributo privado de un objeto.

- Generalmente, el nombre del método comienza con la palabra `get` seguida del nombre del atributo, y su propósito es devolver el valor del atributo al que está asociado.
- Los **getters** suelen ser métodos públicos, lo que permite que otras clases accedan a los valores de los atributos, sin modificar su estado directamente.

```
public class Persona
{
    private String nombre;

    // Getter para el atributo 'nombre'
    public String getNombre() {
           return nombre;
     }
}
```

Clase (setter)

Un setter (también conocido como **mutador**) es un método que se utiliza para **modificar** o **establecer** el valor de un atributo privado de un objeto.

- El nombre del método suele comenzar con `set` seguido del nombre del atributo, y su propósito es asignar un nuevo valor al atributo.
- Los setters proporcionan una manera controlada de modificar los atributos de un objeto, lo que puede incluir lógica adicional, como validación, antes de permitir que el valor se cambie.

```
public class Persona
{
    private String nombre;

    // Setter para el atributo 'nombre'
    public void setNombre(String nombre) {
          this.nombre = nombre;
    }
}
```

Setters y Getters

Esta tendencia que estamos comentando, llevo a Sun Microsystems a definir la especificación de JavaBean. La convención establece que a toda clase que responda a la especificación JavaBean se le requiere:

- A la **clase** se le debe aplicar el modificador de ámbito `public`
- A los **atributos** se le debe aplicar el modificador `private`
- A efectos de proporcionar accesibilidad desde el "exterior" del objeto a dichos atributos, deben de disponer de los correspondientes métodos "**setters**" y "**getters**", que obviamente, también se le debe aplicar el modificador `public.`
- Debe tener un **constructor** sin argumentos
- Debe ser *serializable*. Esto se consigue aplicando la definición de la clase

```
Implements java.io.Serializable
```

Clases envoltorio

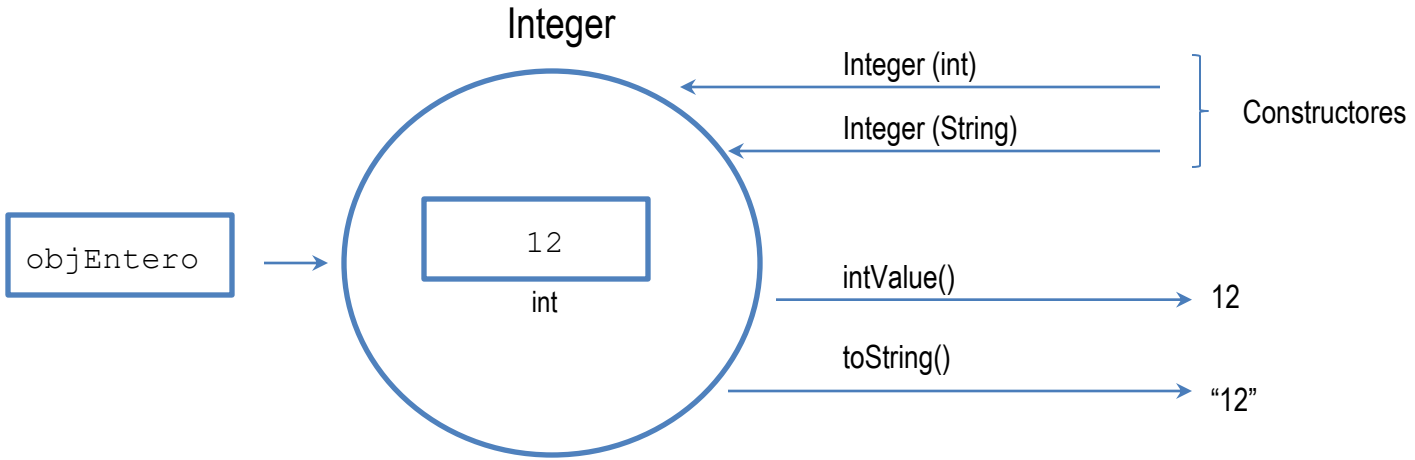
Las **clases envoltorio** o “wrappers”, son clases predefinidas, cuyo fin es aportar encapsulamiento a un tipo primitivo. De tal modo que, un objeto de clase envoltorio, permite acceso mediante sus métodos al dato del tipo primitivo que encapsula. El lenguaje de programación proporciona, y permite tratar, los datos lógicos o booleanos (`boolean`), los datos tipo carácter (`char`), y los numéricos (`byte, shor, int, long, float, double`) como tipo primitivo por cuestiones de eficiencia. Pero hay situaciones o contextos, en que, o bien solamente se permite tratar con objetos, o se recomienda tratar objetos. Es en estos casos o situaciones, en que dado un dato primitivo, lo encapsulamos en su correspondiente clase tipo envoltorio, y ya podemos tratarlo como un objeto. Son estas situaciones que acabamos de describir las que dan sentido de la existencia de las **clases envoltorio**.

```
int primitivoInt = 12;
Integer entero = new Integer(primitivoInt);
```

Clases envoltorio

```
int primitivoInt = 12;
Integer objEntero = new Integer(primitivoInt);
```

Ejercicio:
Probar este código en el IDE y ver que ocurre.



Clases envoltorio

Los métodos de los envoltorios de uso más habitual:

1. Conversión de texto a valores primitivos: Todas las clases envoltorio tienen un método parseXxx (donde Xxx es el nombre del tipo) que convierte un String a su tipo primitivo correspondiente.

```
int numero = Integer.parseInt("42");
double decimal = Double.parseDouble("3.14");
boolean bandera = Boolean.parseBoolean("true");
```

2. Conversión a cadena (String): El método toString() convierte un valor primitivo a su representación en texto.

```
Integer numero = 42;
String texto = numero.toString(); // "42"
```

Autoboxing / Unboxing

Clases envoltorio

3. Constantes útiles:

- MIN_VALUE y MAX_VALUE: Valores mínimo y máximo que puede tomar un tipo primitivo.
- TYPE: Representa el tipo primitivo asociado

```
System.out.println(Integer.MIN_VALUE); // -2147483648
System.out.println(Double.MAX_VALUE); // 1.7976931348623157E308
System.out.println(Character.TYPE);   // char
```

4. Obtención de valores primitivos: Métodos como intValue(), doubleValue(), etc., devuelven el valor primitivo almacenado en el envoltorio.

```
Integer numero = Integer.valueOf(42);
int valorPrimitivo = numero.intValue(); // 42
```

Autoboxing / Unboxing

En Java, puedes convertir automáticamente entre tipos primitivos y sus envoltorios. Esto se conoce como autoboxing (primitivo → objeto) y unboxing (objeto → primitivo).

```
int primitivo = 10;
Integer objeto = primitivo; // Autoboxing
int otroPrimitivo = objeto; // Unboxing
```

Clase STRING

Un objeto de la clase `String`, tiene la finalidad de encapsular un dato tipo cadena de caracteres.
La instanciación de un objeto de la clase `String` puede producirse de dos formas:

1. **Explícita**.- De la misma manera que el resto de clases, es decir, aplicando cualquiera de los constructores disponibles al operador `new`, y con la posibilidad de "recoger" la referencia devuelta por dicho operador al nuevo objeto creado:

```
String cadenaCaracteres = new String ("Esto es una cadena de caracteres");
```

2. **Implícita**.- El hecho de delimitar una cadena de caracteres por " ", implica que se instancie automáticamente un objeto de la clase `String`.

```
String cadenaCaracteres = "Esto es una cadena de caracteres";
```

Clase STRING

Cadena nula

```
String cadenaCaracteres = new String ();
String cadenaCaracteres = "";
```

En ambos casos, se crea el objeto y, por la tanto, existe la cadena nula.

```
String cadenaCaracteres;
String cadenaCaracteres = null;
```

En ambos casos, **NO** se crea el objeto y, por la tanto, no existe la cadena nula, solamente existe la referencia de **cadenaCaracteres**

Clase STRING

A continuación, se muestran los métodos más habituales de uso de la clase `String`:

1. **length()**.- Devuelve la longitud, es decir, el número de caracteres de la cadena encapsulada en el String.

```
String cadenaCaracteres = new String (“Esto es una cadena de caracteres”);
int numeroLongitud = cadenaCaracteres.length();
System.out.println (numeroLongitud);
```

```
32
```

2. **substring(posicion1,posicion2).-** Devuelve la subcadena de caracteres del String comprendida entre el carácter ubicado en la posicion1 (inclusive), hasta el carácter comprendido en la posicion2 (exclusive).

```
String cadenaCaracteres = new String ("Esto es una cadena de caracteres");
String subCadena = cadenaCaracteres.substring(1,4);
System.out.println (subCadena);
```

```
sto
```

Clase STRING

A continuación, se muestran los métodos más habituales de uso de la clase `String`:

3. **indexOf(SubString)**.- Devuelve la posición en que se encuentra la primera ocurrencia del subString transferido como argumento, dentro del String.

```
String cadenaCaracteres = new String ("Esto es una cadena de caracteres");
int valorEncontrado = cadenaCaracteres.indexOf("cadena");
System.out.println (valorEncontrado);
```

```
12
```

4. **charAt(posicion).-** Devuelve el `char` ubicado en la posición transferida como argumento.

```
String cadenaCaracteres = new String ("Esto es una cadena de caracteres");
char charEncontrado = cadenaCaracteres.charAt(3);
System.out.println (charEncontrado);
```

```
o
```

Ejercicio:
Cambiar char por int y observar que ocurre

Clase STRING

A continuación, se muestran los métodos más habituales de uso de la clase `String`:

5. **equals(SubString)**.- Devuelve verdadero o falso, verdadero si se coincide una cadena con otra cadena y falso si no coinciden ambas cadenas.

```
String cadenaCaracteres = new String ("Esto es una cadena de caracteres");
boolean bEncontrado = cadenaCaracteres.equals("caracteres");
System.out.println (bEncontrado);
```

```
False
```

Clase STRINGBUFFER

La concatenación de cadenas consiste en obtener una única cadena resultante a partir de otras. Ello se consigue con el operador “+”, o mediante el método `concat` de la clase String. Veamos el siguiente ejemplo:

```
String cadena1 = " HAZ BIEN ";
cadena1 = cadena1 + "Y NO MIRES CON QUIEN";
System.out.println(cadena1);

String cadena2 = " SON LOBOS ";
cadena2 = cadena2.concat("CON PIEL DE CORDERO");
System.out.println(cadena2);
```

```
 HAZ BIEN Y NO MIRES CON QUIEN
 SON LOBOS CON PIEL DE CORDERO
```

Un objeto de la clase `String` e **inmutable**, es decir, que la cadena que se encapsula cuando se crea no se puede modificar. Lo que se realmente ocurre es que se crea un nuevo objeto de la clase `String`. Dado este mecanismo que acabamos de describir, el hecho de que una cadena original sea sometida a múltiples concatenaciones, puede suponer un importante despilfarro de memoria, y en consecuencia, perdida de eficiencia.

Clase STRINGBUFFER

A efectos de evitar la perdida de eficiencia, se debe utilizar la clase `StringBuffer`. Dicha clase si es **mutable**, a diferencia de `String`, y en consecuencia, el proceso de concatenación de cadenas a una original, no comporta, como en el caso de `String`, la creación de nuevos objetos, sino que el objeto `StringBuffer` original va concatenando tantas cadenas como proceda. Existen 2 métodos para esta realizar el proceso de concatenación con la clase `StringBuffer` (**`append`** e **`insert`**)

```
StringBuffer cadena1 = new StringBuffer();
cadena1.append("HAZ BIEN ");
cadena1.append("Y NO MIRES CON QUIEN ");
System.out.println(cadena1);

StringBuffer cadena2 = new StringBuffer();
cadena2.append("SON LOBOS");
cadena2.insert(2, "Y NO MIRES CON QUIEN ");
System.out.println(cadena2);
```

```
HAZ BIEN Y NO MIRES CON QUIEN
SOY NO MIRES CON QUIEN N LOBOS
```

Método FORMAT

El método **`format`** de Java es parte de la clase **`String`** y se utiliza para crear cadenas formateadas. Es similar a **`printf`** pero devuelve una cadena en lugar de imprimirla directamente. Utiliza especificadores de formato para controlar la representación de valores en la cadena resultante.

```
static String format(formato, objetoAFormatear)
```

Método FORMAT

Características principales del método `format`

- Sintaxis:

```
String.format(String format, Object... args)
```

- Parámetros:
  - **format:** Una cadena que incluye especificadores de formato.
  - **args:** Los valores que serán insertados en los marcadores de formato.

- Especificadores de formato comunes:
  - **%d:** Enteros (decimal).
  - **%f:** Números de punto flotante.
  - **%s:** Cadenas.
  - **%n:** Nueva línea (plataforma independiente).
  - **%t:** Fechas y horas.

Método FORMAT

Ejemplos de uso de `format`

1. Formatear números enteros:

```
int number = 42;
String result = String.format("El número es: %d", number);
System.out.println(result); // Salida: El número es: 42
```

2. Formatear números decimales:

```
double pi = 3.14159;
String result = String.format("El valor de pi es: %.2f", pi); // Solo 2
decimales
System.out.println(result); // Salida: El valor de pi es: 3.14
```

3. Alinear texto:

```
String name = "Juan";
String result = String.format("Hola %-10s!", name); // Alineación a la
izquierda
System.out.println(result); // Salida: Hola Juan      !
```

Método FORMAT

4. Incluir múltiples valores:

```
String name = "Ana";
int age = 25;
String result = String.format("Nombre: %s, Edad: %d", name, age);
System.out.println(result); // Salida: Nombre: Ana, Edad: 25
```

5. Formatear fechas:

```
import java.util.Date;

Date now = new Date();
String result = String.format("Fecha actual: %tF", now); // Formato ISO 8601
System.out.println(result); // Salida: Fecha actual: 2024-12-0
```

6. Formato avanzado con argumentos posicionales

```
String result = String.format("Primero: %1$d, Segundo: %2$s, Primero otra vez: %1$d", 42,
"Hola");
System.out.println(result); // Salida: Primero: 42, Segundo: Hola, Primero otra vez: 42
```

Método FORMAT

Ventajas del método `format`

- **Reutilizable:** Devuelve una cadena, lo que facilita reutilizarla en varias partes del programa.
- **Legible:** Hace que el código sea más comprensible en comparación con concatenaciones largas.
- **Control sobre el formato:** Permite una personalización precisa en la presentación de datos.

Clases Encapsuladoras de Fechas

En Java, las clases encapsuladoras de fechas se encuentran principalmente en los paquetes **`java.time`** (introducido en Java 8 como parte de la API moderna de fecha y hora) y **`java.util`** (la API anterior). La API moderna es preferida debido a su diseño robusto, claridad, y manejo explícito de zonas horarias y formatos.

Clases Encapsuladoras de Fechas

Ejemplos de uso de `java.time`

1. **`LocalDate`** (Representa una fecha sin hora ni zona horaria):

```
import java.time.LocalDate;

public class Main {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now(); // Fecha actual
        LocalDate birthDate = LocalDate.of(1990, 5, 15); // Fecha específica

        System.out.println("Hoy: " + today); // Salida: Hoy: 2024-12-06
        System.out.println("Cumpleaños: " + birthDate); // Salida: Cumpleaños: 1990-05-15
    }
}
```

Clases Encapsuladoras de Fechas

2. **`LocalTime`** (Representa una hora sin información de fecha ni zona horaria):

```
import java.time.LocalTime;

public class Main {
    public static void main(String[] args) {
        LocalTime now = LocalTime.now(); // Hora actual
        LocalTime meetingTime = LocalTime.of(15, 30); // 15:30

        System.out.println("Hora actual: " + now); // Salida: Hora actual: 14:45:20.123
        System.out.println("Reunión: " + meetingTime); // Salida: Reunión: 15:30
    }
}
```

Clases Encapsuladoras de Fechas

3. **`LocalDateTime`** (Combina fecha y hora sin zona horaria):

```
import java.time.LocalDateTime;

public class Main {
    public static void main(String[] args) {
        LocalDateTime now = LocalDateTime.now();
        LocalDateTime event = LocalDateTime.of(2024, 12, 25, 18, 0);

        System.out.println("Ahora: " + now); // Salida: Ahora: 2024-12-06T14:45:20.123
        System.out.println("Evento: " + event); // Salida: Evento: 2024-12-25T18:00
    }
}
```

Clases Encapsuladoras de Fechas

4. **`ZonedDateTime`** (Incluye fecha, hora y zona horaria):

```
import java.time.ZonedDateTime;
import java.time.ZoneId;

public class Main {
    public static void main(String[] args) {
        ZonedDateTime now = ZonedDateTime.now(); // Fecha y hora actual con zona local
        ZonedDateTime tokyoTime = ZonedDateTime.now(ZoneId.of("Asia/Tokyo")); // Zona específica

        System.out.println("Ahora: " + now); // Ahora: 2024-12-
06T18:31:14.660484200+01:00[Europe/Madrid]
        System.out.println("Hora en Tokio: " + tokyoTime); // Hora en Tokio: 2024-12-
07T02:31:14.673965+09:00[Asia/Tokyo]
    }
}
```

Clases Encapsuladoras de Fechas

5. **`Period`** (Representa un intervalo de tiempo en años, meses y días):

```
import java.time.LocalDate;
import java.time.Period;

public class Main {
    public static void main(String[] args) {
        LocalDate start = LocalDate.of(2020, 1, 1);
        LocalDate end = LocalDate.of(2024, 12, 6);
        Period period = Period.between(start, end);

        System.out.println("Años: " + period.getYears()); // Años: 4
        System.out.println("Meses: " + period.getMonths()); // Meses: 11
        System.out.println("Días: " + period.getDays()); //Días: 5
    }
}
```

Clases Encapsuladoras de Fechas

6. **`Duration`** (Intervalo en horas, minutos y segundos):

```
import java.time.Duration;
import java.time.LocalTime;

public class Main {
    public static void main(String[] args) {
        LocalTime start = LocalTime.of(10, 0);
        LocalTime end = LocalTime.of(14, 30);
        Duration duration = Duration.between(start, end);

        System.out.println("Duración: " + duration.toHours() + " horas"); // Duración: 4 horas
    }
}
```

T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
6.3.1. Introducción
6.3.2. Ámbitos Private / Public / Protected / Por Defecto
6.3.3. Getters & Setters
6.3.4. Clases envoltorio
6.3.5. Clase STRING
6.3.5. Clase STRINGBUFFER
6.3.6. Método FORMAT
6.3.7. Clases Encapsuladoras de Fechas
6.4. Herencia
6.4.1. Introducción
6.4.2. Extends
6.4.3. Sobreescritura de métodos: SUPERCLASE
6.4.4. Interface
6.4.5. Abstract
6.4.6. Final
6.4.7. Herencia Simple
6.5. Polimorfismo
6.5.1. Introducción
6.5.2. Tipos de polimorfismo
6.6. Lenguaje Unificado de Modelado (UML)
6.6.1 Diagramas de clases

6.4. Herencia

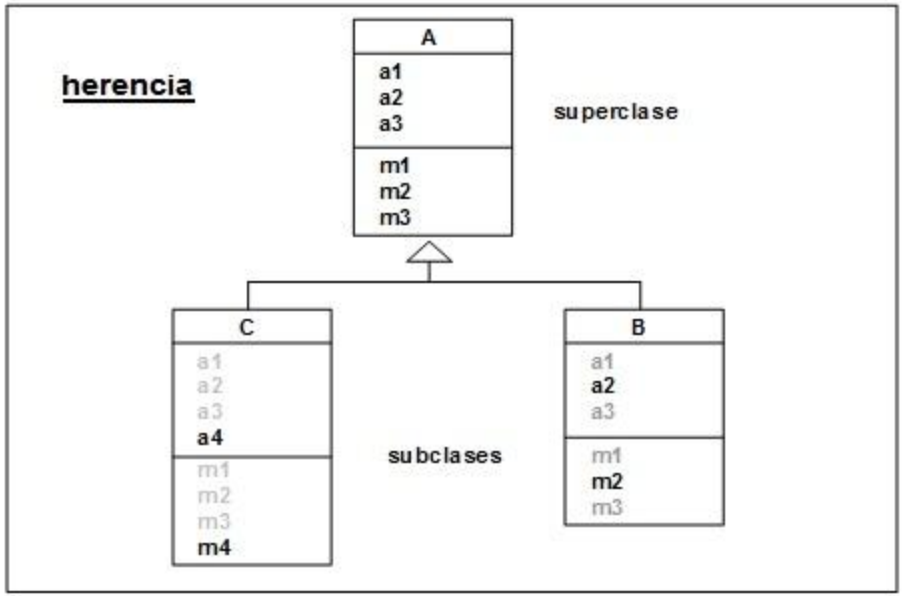
Herencia

La **herencia** en Programación Orientada a Objetos (POO) es un mecanismo que permite crear una nueva clase (denominada clase derivada o **subclase**) basándose en una clase existente (denominada clase base o **superclase**). La subclase hereda los atributos y métodos de la superclase, lo que fomenta la reutilización del código y establece una relación jerárquica entre las clases.

Características principales de la herencia:

- **Reutilización de código:** La subclase puede usar los métodos y atributos definidos en la superclase sin necesidad de volver a implementarlos.
- **Extensibilidad:** La subclase puede añadir sus propios métodos y atributos adicionales para extender la funcionalidad de la superclase.
- **Sobrescritura (Override):** Las subclases pueden redefinir los métodos de la superclase para personalizar su comportamiento.
- **Polimorfismo:** La herencia facilita el polimorfismo, permitiendo que un objeto de una subclase se trate como si fuera de la superclase.

Herencia



En la imagen, la clase B comparte las propiedades de la clase A, pero redefine a2 y m2. La clase C, también, comparte los atributos de la clase A, pero añade las propiedades a4 y m4. Las subclases no pueden rechazar ninguna propiedad de sus superclases. Se acostumbra a escribir sólo lo nuevo y se omiten las propiedades sin cambio. La relación de herencia se establece desde las subclases hacia la superclase. Es decir, B y C indican explícitamente que son subclases de A en cada una de sus declaraciones. La palabra **subclase** sólo denota que es una clase que hereda y la palabra **superclase** sólo denota que es una clase de la que se hereda. La superclase y la subclase reciben también reciben otros nombres, por ejemplo: clase base y clase derivada respectivamente.

Extends

```
// Superclase
public class Vehiculo{
    String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void arra
        System.out.p
motor.");
    }
}
```

```
public class Coche extends Vehiculo {
    int puertas;

    public Coche(String marca, int puertas) {
        super(marca); // Llama al atributo de la superclase
        this.puertas = puertas;
    }

on " +
```

```
public class Main {
    public static void main(String[] args) {
        // Crear un objeto de la superclase
        Vehiculo objVehiculo = new Vehiculo("Vehículo genérico");
        objVehiculo.arrancarMotor(); // Salida: Vehículo genérico está arrancando el motor.

        // Crear un objeto de la subclase
        Coche objCoche = new Coche("Toyota", 4);
        objCoche.arrancarMotor(); // Salida: Toyota está arrancando el motor.
        objCoche.mostrarInfo(); // Salida: Este es un Toyota con 4 puertas.
    }
}
```

La impleme
clase espe
continuació

reda" de la
ue figura a

Extends

```
public class Animal {
    String nombre;

    public Animal(Str
        this.nombre =
    }

    public void hacer
        System.out.pr
    }
}
```

```
public class Perro extends Animal {

    public Perro (String nombre) {
        super(nombre); // Llama al atributo de la superclase
    }

    // Sobrescritura de un método
```

```
public class Main {
    public static void main(String[] args) {
        Animal objAnimal = new Animal("Animal genérico");
        objAnimal.hacerSonido(); // Salida: El animal hace un sonido.

        Perro objPerro = new Perro("Rocky");
        objPerro.hacerSonido(); // Salida: Rocky dice: ¡Guau!
        objPerro.mostrarInfo(); // Salida: Rocky está trayendo la pelota.

        // Polimorfismo: tratar un Perro como un Animal
        Animal objOtroPerro = new Perro("Rex");
        objOtroPerro.hacerSonido(); // Salida: Rex dice: ¡Guau!
        // objOtroPerro.mostrarInfo(); // Error: mostrarInfo no está definido en la clase Animal
    }
}
```

```
pelota.");
```

Herencia

Ventajas de la herencia:

- Reutilización: Reduce la redundancia en el código.
- Modularidad: Mejora la estructura del programa dividiendo responsabilidades.
- Extensibilidad: Facilita la adición de nuevas funcionalidades.

Limitaciones de la herencia:

- Alto acoplamiento: La subclase depende de la implementación de la superclase, lo que puede complicar los cambios.
- No reutilizable para relaciones no jerárquicas: Cuando dos clases no tienen una relación de "es-un(a)", la herencia puede ser inapropiada.

Cuándo usar herencia:

- Cuando existe una relación lógica de "es-un(a)" (e.g., un perro es un animal).
- Si varias clases comparten atributos y comportamientos comunes.

Sobreescritura de métodos: SUPERCLASE

La **sobreescritura** de métodos (method overriding) en Java es una característica de la herencia que permite a una subclase proporcionar una implementación específica de un método que ya está definido en su superclase. Esto se utiliza para personalizar o extender el comportamiento heredado.

Sobreescritura de métodos: SUPERCLASE

Características clave de la sobrescritura de métodos:

1. Mismo nombre, parámetros y tipo de retorno:

- El método en la subclase debe tener la misma firma (nombre y parámetros) y tipo de retorno que el de la superclase.

2. Relación jerárquica:

- La sobrescritura ocurre solo entre una clase base (superclase) y una subclase.

3. Uso del modificador @Override:

- Aunque es opcional, se recomienda usar la anotación @Override para indicar que un método está siendo sobrescrito. Esto ayuda a evitar errores como cambios accidentales en la firma del método.

4. Invocar el método de la superclase:

- La palabra clave super permite llamar al método original de la superclase desde la subclase.

SUPERCLASE

```
public class Animal {
    String nombre;

    public Animal(String nombre) {
        this.name = n
    }

    public void hacer
        System.out.pr
    }
}
```

```
public class Perro extends Animal {

    public Perro (String nombre) {
        super(nombre); // Llama al constructor de la superclase
    }

    // Sobrescritura de un método
    @Override
```

```
public class Main {
    public static void main(String[] args) {
        Animal objAnimal = new Animal("Animal genérico");
        objAnimal.hacerSonido(); // Salida: El animal hace un sonido.

        Perro objPerro = new Perro("Rocky");
        objPerro.hacerSonido(); // Salida: Rocky dice: ¡Guau!
        objPerro.mostrarInfo(); // Salida: Rocky está trayendo la pelota.

        // Polimorfismo: tratar un Perro como un Animal
        Animal objOtroPerro = new Perro("Rex");
        objOtroPerro.hacerSonido(); // Salida: Rex dice: ¡Guau!
        // objOtroPerro.mostrarInfo(); // Error: mostrarInfo no está definido en la clase Animal
    }
}
```

```
elota.");
```

Interface

Una **`interface`** es un tipo superior jerárquico en que se definen "cabeceras" de métodos, sin ninguna implementación de código. Las clases se definen como subclases de la **`interface`** mediante **`implements`**

```
public class SubClase1 implements Superior
```

Deberán definir e implementar, obligatoriamente, todos los métodos cuyas "cabeceras" han sido definidas en la **`interface`**. Es decir, que el hecho de que una clase herede de una **`interface`**, implica que dicha clase asume una obligación de definir e implementar todos los métodos cuyas "cabeceras" constan en dicha superclase. Dicho de otro modo, someterse a una "disciplina".

Interface

Ejercicio: Probar a ampliar a void frenar()

```
public interface Conducible {
    void conducir();
}
```

```
public class Coche implements Conducible {
    @Override
    public void conducir() {
        System.out.println("El coche está siendo
conducido.");
    }
}
```

```
public class Bicicleta implements Conducible {
    @Override
    public void conducir() {
        System.out.println("La bicicleta está siendo conducida.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // POLIMORFISMO
        Conducible coche = new Coche();
        Conducible bicicleta = new Bicicleta();

        coche.conducir();       // Salida: El coche está siendo conducido.
        bicicleta.conducir();   // Salida: La bicicleta está siendo
conducida.
    }
}
```

Abstract

La palabra clave **`abstract`** en Java se usa para declarar algo que es incompleto o genérico. Esto puede aplicarse a clases o métodos.

En otras palabras, cuando algo es **`abstract`**, significa que no está completamente definido, y necesita ser completado por una clase o método específico.

Sirve para definir atributos y métodos comunes que serán compartidos por sus clases hijas.

Abstract

Ejemplo

```java
public abstract class Animal {
    String nombre;

    // Método común para todas las clases hijas
    public void respirar() {
        System.out.println(nombre + " está respirando.");
    }

    // Método abstracto (no tiene cuerpo)
    public abstract void hacerSonido();
}
```

```java
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El perro hace: ¡Guau!");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        // No puedes hacer esto: Animal animal = new Animal(); (da error)

        Animal perro = new Perro(); // Polimorfismo
        perro.hacerSonido(); // Salida: El perro hace: ¡Guau!
    }
}
```

Abstract

¿Por qué usar abstract?

- Generalizar comportamientos: Sirve para definir métodos que serán comunes a varias clases, pero cuya implementación será diferente en cada caso.

- Organizar el código: Las clases abstractas ayudan a estructurar jerarquías lógicas de clases (e.j., todos los animales respiran, pero no todos hacen el mismo sonido).

- Evitar duplicación: Los métodos concretos en la clase abstracta evitan que las clases hijas tengan que implementar comportamiento repetido.

Abstract

Resumen:

- abstract en clases: Es una clase que no se puede instanciar. Sirve como base para otras clases que completan su funcionalidad.

- abstract en métodos: Es un método sin cuerpo que debe ser implementado por las clases hijas.

- Ejemplo práctico: Animal puede ser abstracto porque no todos los animales son iguales, pero todos comparten algunas características (como respirar).

Diferencia entre interface y abstract

Aspecto	Clase Abstracta	Interfaz
Definición	Puede contener **métodos abstractos** (sin cuerpo) y **métodos concretos** (con implementación).	Solo contiene **métodos abstractos** (hasta Java 7). A partir de Java 8, puede incluir métodos con implementación mediante default y métodos estáticos.
Modificadores de acceso	Los métodos pueden tener **cualquier modificador de acceso** (e.g., public, protected, private).	Los métodos son implícitamente **públicos** (a menos que sean default o static).
Propósito	Representa **una jerarquía de clases** y define un comportamiento común para las subclases.	Define **un contrato puro** que las clases deben implementar, independientemente de la relación jerárquica.
Herencia múltiple	Una clase solo puede heredar de una clase abstracta (**herencia simple**).	Una clase puede implementar múltiples interfaces, lo que permite **herencia múltiple** de comportamiento.
Constructores	**Puede tener constructores** porque forma parte de una jerarquía de clases.	**No puede tener constructores**, ya que no se pueden crear instancias directamente de una interfaz.
Atributos	**Puede tener atributos** (variables de instancia) con cualquier modificador de acceso y pueden ser modificables.	**Solo puede tener constantes** (static final) que deben ser inicializadas.
Compatibilidad	Útil para compartir código común entre clases relacionadas.	Más flexible, ya que permite que clases no relacionadas compartan un comportamiento común.

Final

El modificador final es aplicable a clases, a métodos y a atributos, de tal modo que en cada uno de estos casos tiene diferente repercusión.

- clase: Impide que pueda actuar como superclase de otras, es decir, ninguna otra clase puede heredar de ella, convirtiéndose en subclase de sí misma.

```
public final class Circulo {
    // ...
}
```

Final

El modificador final es aplicable a clases, a métodos y a atributos, de tal modo que en cada uno de estos casos tiene diferente repercusión.

- método: Este caso es aplicable a un método definido en una superclase. Dicho modificador impide que en las subclases puedan sobrescribir dichos métodos. Solo pueden ser heredados implícitamente.

```
public class Animal {
    public final void respirar() {
            System.out.println("Respirando...");
    }
}
```

Final

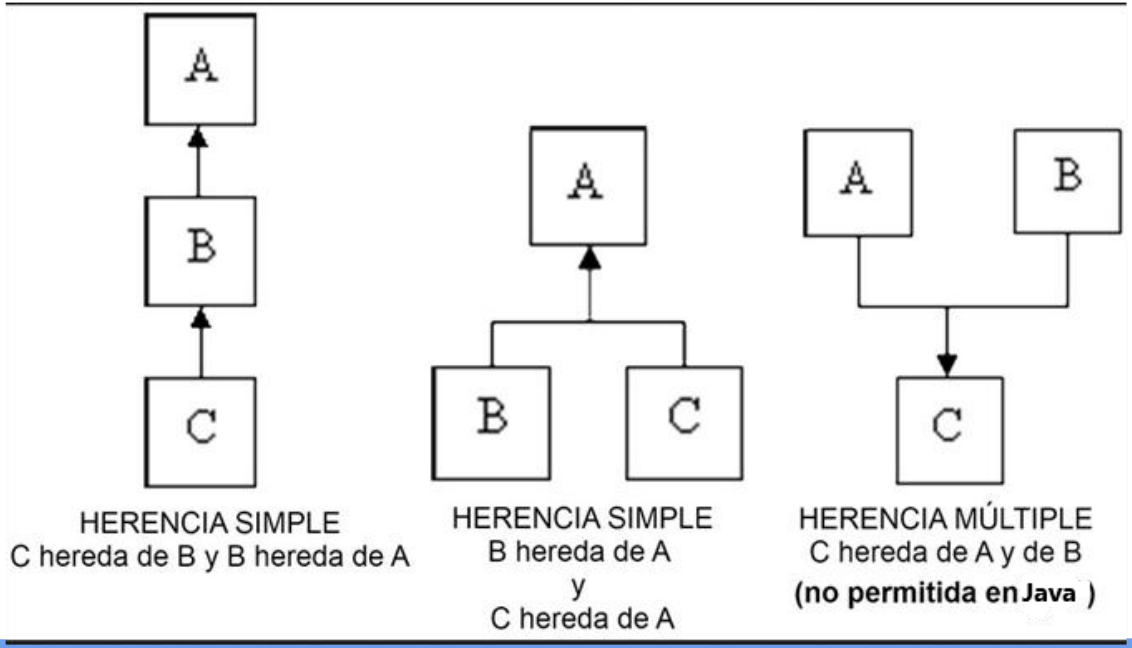
El modificador final es aplicable a clases, a métodos y a atributos, de tal modo que en cada uno de estos casos tiene diferente repercusión.

- atributo: Lo convierte en una constante.

```
public class Persona {
    private final String nombre;
    public Persona(String nombre) {
         this.nombre = nombre;
    }
}
```

Herencia simple

La **herencia** simple en Java es un concepto fundamental de la Programación Orientada a Objetos (POO). Permite que una clase herede atributos y métodos de otra clase. Es como si una clase "hija" tomara las características y comportamientos de una clase "padre".



T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
6.3.1. Introducción
6.3.2. Ámbitos Private / Public / Protected / Por Defecto
6.3.3. Getters & Setters
6.3.4. Clases envoltorio
6.3.5. Clase STRING
6.3.5. Clase STRINGBUFFER
6.3.6. Método FORMAT
6.3.7. Clases Encapsuladoras de Fechas
6.4. Herencia
6.4.1. Introducción
6.4.2. Extends
6.4.3. Sobreescritura de métodos: SUPERCLASE
6.4.4. Interface
6.4.5. Abstract
6.4.6. Final
6.4.7. Herencia Simple
6.5. Polimorfismo
6.5.1. Introducción
6.5.2. Tipos de polimorfismo
6.6. Lenguaje Unificado de Modelado (UML)
6.6.1 Diagramas de clases

6.5. Polimorfismo

Polimorfismo

La palabra "**polimorfismo**" significa "muchas formas". En el contexto de Java, el **polimorfismo** permite que un mismo método o acción se comporte de diferentes formas, dependiendo del objeto que lo esté utilizando. Es una forma de adaptar el comportamiento de una clase o un método sin cambiar su interfaz.

Tipos de polimorfismo

Polimorfismo en tiempo de compilación (Sobrecarga de métodos):

- Ocurre cuando se definen varios métodos con el mismo nombre, pero con diferentes parámetros.
- La decisión de qué método se usa se toma durante la compilación.

Polimorfismo en tiempo de ejecución (Sobreescritura de métodos):

- Ocurre cuando una subclase sobrescribe un método definido en la clase padre.
- La decisión de qué método se usa se toma durante la ejecución.

Tipos de polimorfismo

Ejemplo práctico: Polimorfismo en tiempo de ejecución (Sobreescritura de métodos)

Clase padre

```
public class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido.");
    }
}
```

Clase principal

```
public class Main {
    public static void main(String[] args) {
        Animal miAnimal;

        miAnimal = new Perro();
        miAnimal.hacerSonido();  // Salida: El perro hace: ¡Guau!

        miAnimal = new Gato();
        miAnimal.hacerSonido();  // Salida: El gato hace: ¡Miau!
    }
}
```

Clase hijos

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El perro hace: ¡Guau!");
    }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El gato hace: ¡Miau!");
    }
}
```

Tipos de polimorfismo

Ejemplo práctico: Polimorfismo en tiempo de compilación (Sobrecarga de métodos)

Sobrecarga de métodos

```
public class Calculadora {
    // Método para sumar dos números
    public int sumar(int a, int b) {
        return a + b;
    }

    // Método para sumar tres números
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }
}
```

Clase principal

```
Public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();

        System.out.println(calc.sumar(2, 3));        // Salida: 5
        System.out.println(calc.sumar(2, 3, 4));    // Salida: 9
    }
}
```

T6. Principios fundamentales de la POO

6.1. Introducción
6.2. Abstracción
6.3. Encapsulamiento
- 6.3.1. Introducción
- 6.3.2. Ámbitos Private / Public / Protected / Por Defecto
- 6.3.3. Getters & Setters
- 6.3.4. Clases envoltorio
- 6.3.5. Clase STRING
- 6.3.5. Clase STRINGBUFFER
- 6.3.6. Método FORMAT
- 6.3.7. Clases Encapsuladoras de Fechas

6.4. Herencia
- 6.4.1. Introducción
- 6.4.2. Extends
- 6.4.3. Sobreescritura de métodos: SUPERCLASE
- 6.4.4. Interface
- 6.4.5. Abstract
- 6.4.6. Final
- 6.4.7. Herencia Simple

6.5. Polimorfismo
- 6.5.1. Introducción
- 6.5.2. Tipos de polimorfismo

6.6. Lenguaje Unificado de Modelado (UML)
- **6.6.1 Diagramas de clases**

6.6. Lenguaje Unificado de Modelado (UML)

Lenguaje Unificado de Modelado (UML)

El Lenguaje Unificado de Modelado (UML) es un estándar visual para representar, diseñar y entender sistemas de software, especialmente en la Programación Orientada a Objetos (POO). Es como un lenguaje universal que los desarrolladores, analistas y diseñadores utilizan para comunicar ideas y entender cómo funciona o debe funcionar un sistema.

Lenguaje Unificado de Modelado (UML)

¿Por qué usar UML?

- Comunicación: Ayuda a que los equipos entiendan mejor cómo se estructura y funciona un programa.
- Diseño: Permite planificar el sistema antes de escribir el código.
- Documentación: Sirve como guía para mantener el software y para nuevos desarrolladores.
- Visualización: Hace más fácil ver cómo se relacionan las clases, objetos y procesos.

Lenguaje Unificado de Modelado (UML)

¿Qué tipos de diagramas tiene UML?

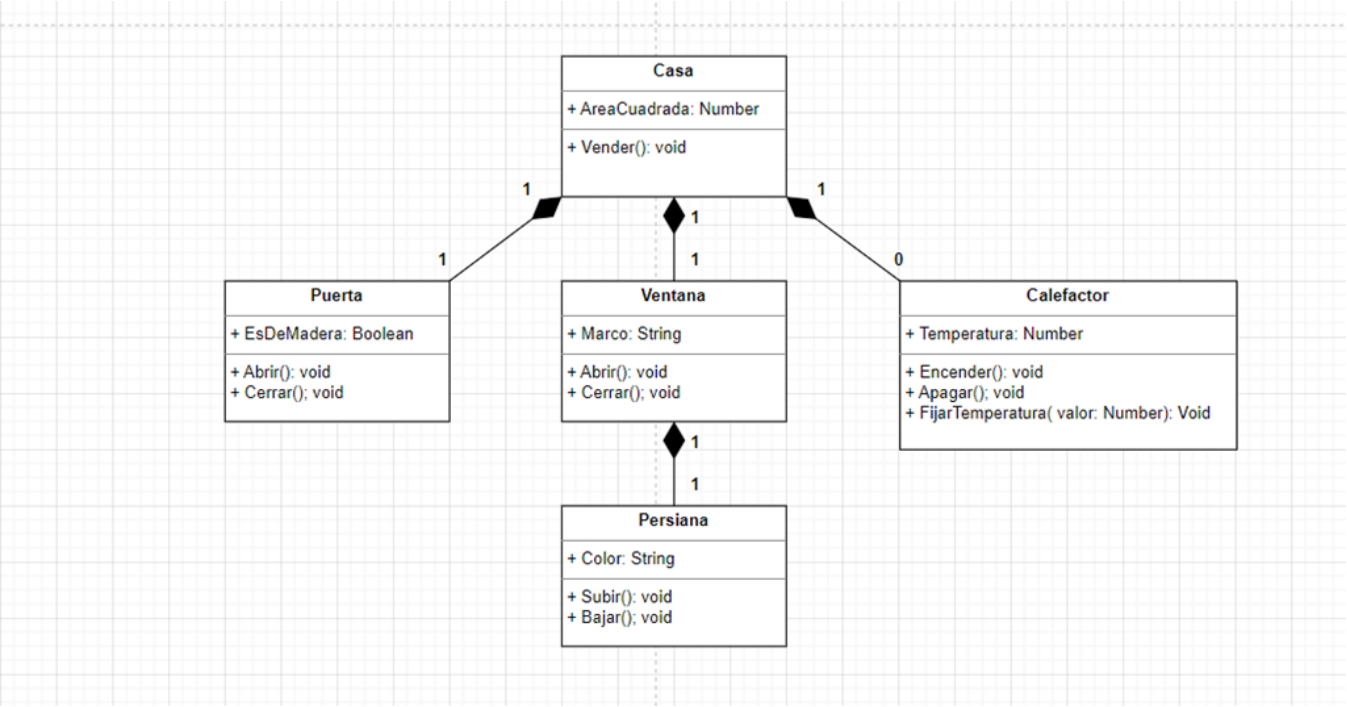
UML incluye varios tipos de diagramas, pero los principales (y más útiles para DAM) son:

1. **Diagrama de Clases:**

- Propósito: Muestra la estructura del sistema, las clases, sus atributos, métodos y cómo se relacionan.
- Útil para: Representar la POO.
- Ejemplo: Clase Coche con atributos como marca y modelo y métodos como acelerar().

Lenguaje Unificado de Modelado (UML)

1. Diagrama de Clases:



https://www.youtube.com/watch?v=Z0yLerU0g-Q

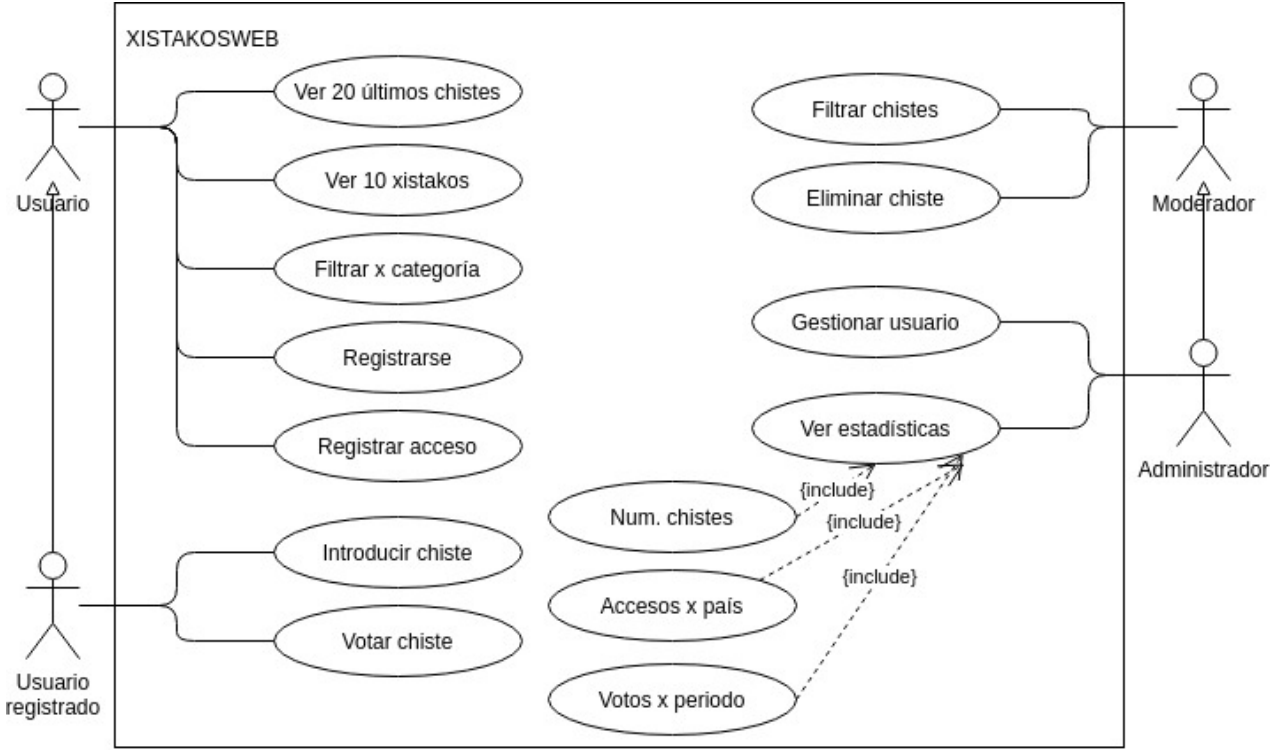
Lenguaje Unificado de Modelado (UML)

2. Diagrama de casos de uso:

- Propósito: Representa qué puede hacer el usuario en el sistema (interacciones entre usuarios y el software).
- Útil para: Entender los requisitos funcionales.
- Ejemplo: Un usuario puede "Iniciar sesión" y "Consultar saldo" en un sistema bancario.

Lenguaje Unificado de Modelado (UML)

2. Diagrama de casos de uso:



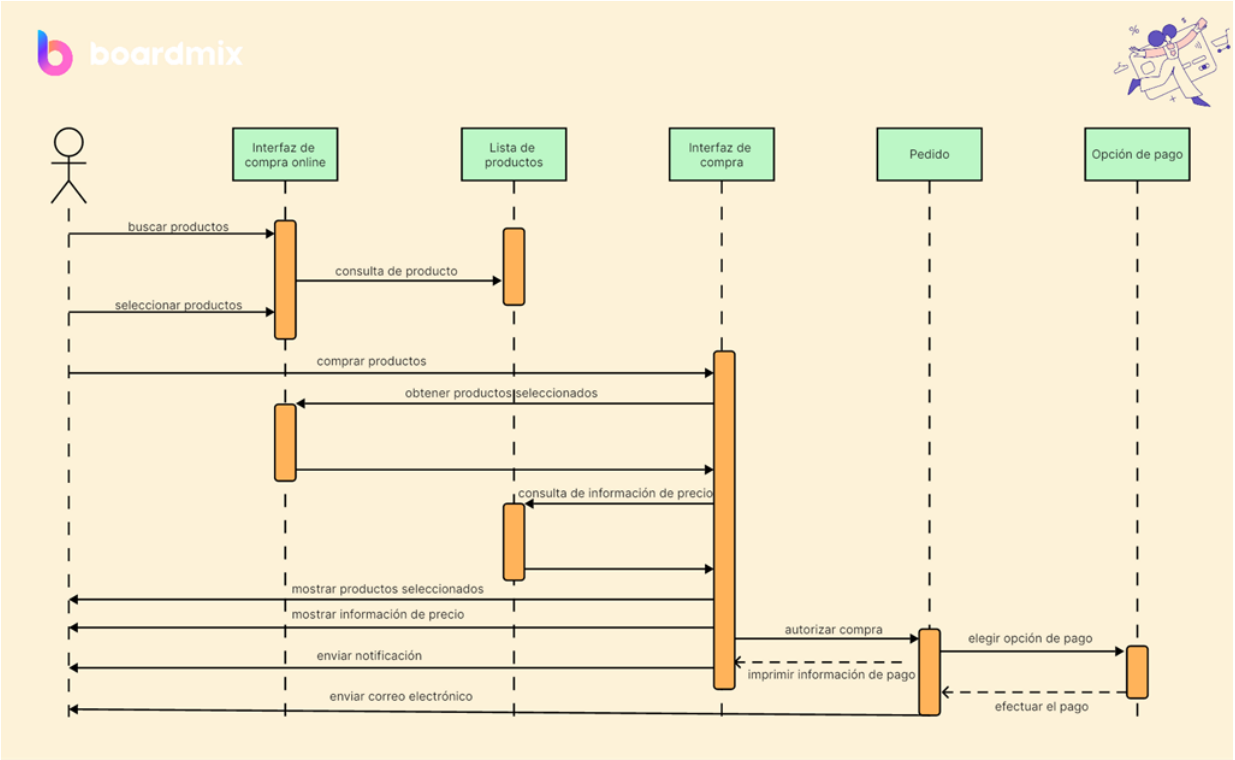
Lenguaje Unificado de Modelado (UML)

3. Diagrama de Secuencia

- Propósito: Muestra cómo interactúan los objetos y en qué orden ocurre cada acción
- Útil para: Entender el flujo de eventos.
- Ejemplo: Un usuario solicita información de su saldo al servidor y recibe una respuesta..

Lenguaje Unificado de Modelado (UML)

3. Diagrama de Secuencia



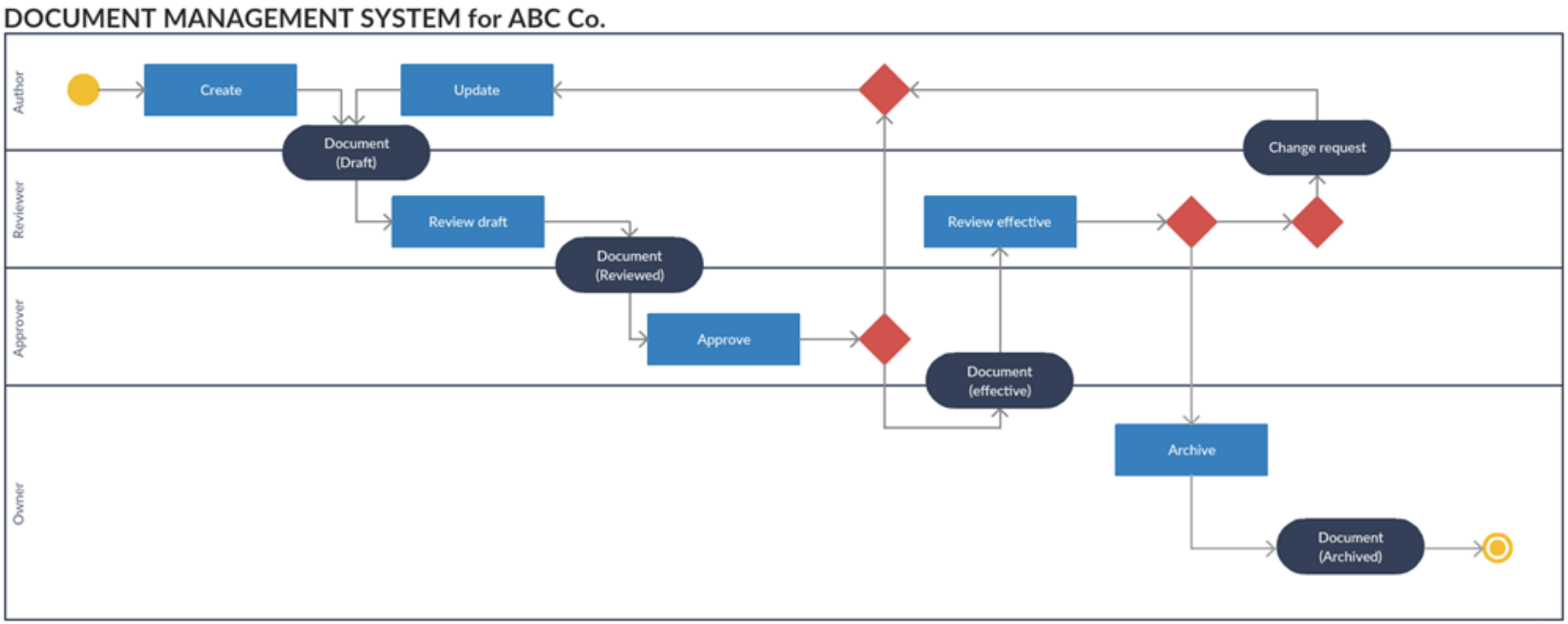
Lenguaje Unificado de Modelado (UML)

4. Diagrama de Actividades

- Propósito: Representa el flujo de trabajo o los procesos.
- Útil para: Modelar algoritmos o flujos de procesos.
- Ejemplo: Comprar un producto:
    - Seleccionar producto -> Pagar -> Confirmar envío.

Lenguaje Unificado de Modelado (UML)

4. Diagrama de Actividades



Grado Superior

Programación

T6. Principios fundamentales de la POO

Curso 2025/26

Profesor

Raúl Murillo Gallego

«Rodéate de personas que crean en tus sueños, animen tus ideas, apoyen tus ambiciones, y saquen lo mejor de ti» —Roy T. Bennet